

引用格式: 郝鹏涛, 肖锋, 黄姝娟, 等. 基于通信开销的多类型 DAG 任务调度方法[J]. 微电子学与计算机, 2024, 41(5): 67-75.
HAO P T, XIAO F, HUANG S J, et al. The multi-type DAG task scheduling method based on communication overhead[J].
Microelectronics & Computer, 2024, 41(5): 67-75.
DOI: [10.19304/J.ISSN1000-7180.2023.0135](https://doi.org/10.19304/J.ISSN1000-7180.2023.0135)

基于通信开销的多类型 DAG 任务调度方法

郝鹏涛¹, 肖 锋¹, 黄姝娟¹, 张文娟², 陈雯琦¹, 左俊杰¹

(1 西安工业大学 计算机科学与工程学院, 陕西 西安 710021;

2 西安工业大学 基础学院, 陕西 西安 710021)

摘 要: 随着异构多核处理器的性能不断增强, 具有关联关系任务的调度问题和调度算法成为研究者们关注的焦点。针对异构多核平台上的有向无环图(Directed Acyclic Graph, DAG)任务模型进行了调度分析。由于异构环境中计算资源类型丰富数量多, 实际调度过程中处在不同核上的任务之间的通信开销是存在的。而当前大多数调度算法要么忽略不同核上的任务之间的通信开销, 要么优先级判定和选择处理器核执行时不能全面考虑核间任务的通信开销, 造成资源浪费, 增加了系统调度任务的整体长度。因此, 针对具有通信开销的 DAG 任务模型, 讨论了任务间的通信开销对任务调度算法的影响, 提出了复合节点生成算法(Composite Node Generation Algorithm, CNGA)任务调度算法。该算法根据任务间通信开销大小进行预处理分配生成复合节点, 重构 DAG 图将任务节点的上行秩和通信占比作为参数决定任务节点的优先级。实验结果与 HEFT 和 CPOP 算法比较, 在任务通信开销和任务执行开销之比(Communication to Computation Ratio, CCR)一定的情况下响应时间比原有的 HEFT 和 CPOP 提升 10.19%, 稳定性以松弛度和接受率为参数平均提升 3.96%。

关键词: DAG; 任务调度; 通信开销; 复合节点; CNGA 算法

中图分类号: TP312

文献标识码: A

文章编号: 1000-7180(2024)05-0067-09

The multi-type DAG task scheduling method based on communication overhead

HAO Pengtao¹, XIAO Feng¹, HUANG Shujuan¹, ZHANG Wenjuan²,
CHEN Wenqi¹, ZUO Junjie¹

(1 School of Computer Science and Engineering, Xi'an Technological University, Xi'an 710021, China;

2 School of Science, Xi'an Technological University, Xi'an 710021, China)

Abstract: With the increasing performance of heterogeneous multi-core processors, the scheduling problem and scheduling algorithm with relational tasks become the focus of researchers. In this paper, the Directed Acyclic Graph (DAG) task model on heterogeneous multi-core platform is analyzed. Because there are many types of computing resources in heterogeneous environment and they are widely distributed, the communication overhead between tasks on different cores exists in the actual scheduling process. However, most of the current scheduling algorithms either ignore the communication overhead between tasks on different cores, or fail to consider the communication overhead of tasks between cores when deciding and selecting processor cores, resulting in waste of resources and increasing the overall length of the system scheduling tasks. Therefore, for the DAG task model with communication overhead, this paper discusses the

收稿日期: 2023-02-21; 修回日期: 2023-04-28

基金项目: 国家自然科学基金面上项目(62171361); 陕西省重点研发计划一般项目(2022GY-119); 陕西省科技厅自然科学基础研究计划(2021JM-440)

<http://www.journalmc.com>

influence of the communication overhead between tasks on the task scheduling algorithm, and proposes a Composite Node Generation Algorithm(CNGA) task scheduling algorithm. According to the amount of communication overhead between tasks, the algorithm is preprocessed and allocated to generate composite nodes, and the upstream rank and communication ratio of task nodes are used as parameters to determine the priority of task nodes. Compared with HEFT and CPOP algorithms, the experimental results show that the ratio of task communication overhead to task execution overhead Communication to Computation Ratio (CCR) under certain conditions, the response time is improved by 10.19% compared with the original HEFT and CCOP, and the stability is increased by 3.96%.

Key words: DAG; task scheduling; communication overhead; compound node; CNGM algorithm

1 引言

异构多核环境下的任务调度是 NP 完全问题^[1], 任务调度的最终目的只能获取近似的最优解。在异构多核环境下, 任务的调度情况比同构环境下考虑的因素更为复杂。因此需要良好的调度决策以提升处理器系统的性能^[2]和系统运行效率^[3]。国外学者 Topcuolu 等^[4]在 2002 年提出了异构分布式计算系统下有向无环图(Directed Acyclic Graph, DAG)调度模型, 其中具有关联关系的任务调度模型用 DAG 图来表示出任务调度过程中的抽象逻辑。在任务调度执行过程中需要考虑两种不同类型的开销: 执行任务的开销和任务之间的通信开销^[5]。为了达到提升处理器性能的目的, 调度算法需要满足: 任务间通信最小化以及任务调度的高度并行性。在 DAG 任务调度过程中, 如果为了消除任务间通信开销, 将所有任务放在同一处理器核上执行会导致负载不均衡^[6]。另一方面处理器内核按照执行开销平均分配任务会使各个内核之间负载均衡, 但任务的频繁切换会增加系统整体的通信开销从而增加任务的调度长度。在任务调度过程中需要兼顾这两方面因素, 使任务调度能够充分发挥多核系统的优势。

本文针对带通信开销的多类型 DAG 任务进行研究, 提出一种新的复合节点生成算法(Composite Node Generation Algorithm, CNGA)任务调度算法, 通过 DAG 任务模型的预处理, 将可在同一内核上调度的节点合并生成复合节点, 转化后的 CD-DAG(Cool Down DAG)任务模型相比之前的模型可以有效减少不必要的通信开销。同时为避免生成复合节点过大造成内核负载过大, 在调度时设置好阈值用于限制生成复合节点的原子节点的数量。

2 相关工作

多核异构系统任务调度问题近年来引起了国内外研究学者们的广泛关注, 学者们针对不同类型的任务模型在不同资源环境下调度的相关问题展开了

<http://www.journalmc.com>

研究。

Topcuolu 等^[4]提出的 HEFT 算法是使用广泛的列表调度算法, HEFT 算法使用任务的平均计算成本和平均通信成本计算任务的 $rank_u$ ^[4]用来代表当前任务到出口任务的期望路径长度, $rank_u$ 值越大, 任务的优先级越高。在处理器选择阶段, HEFT 算法每次选择 $rank_u$ 最大的任务并将其分配到具有最早完成时间的处理器上, 并且在任务分配时考虑处理器上是否有足够的空闲时隙完成任务的插入。HEFT 算法模型在调度过程中屏蔽了复杂的具体细节, 在任务调度的精确性和简单性之间做到了平衡。然而这种算法在具体调度的过程中效率过低。

针对 HEFT 中的插入策略进行了优化, Munir 等^[7]提出了 SDBTAS 算法, 以及 Wang 等^[8]在计算任务优先级时引入计算成本的标准差代替平均值提出的 HSIP 算法。除此之外, 还有 CPOP 算法^[4]、MCP 算法^[7]以及 LDCP 算法^[7], 其主要核心都是 DAG 任务调度图中关键路径上的任务, 其中关键路径定义为 DAG 图中距离最长的路径, 代表了调度时间长度的下限。但由于调度中不断迭代更新后继路径的关键节点会出现空闲的时间碎片, 造成整体调度时间的增大, 调度过程中未考虑核间通信开销所带来的影响。

Bittencourt 等^[9]在 HEFT 算法的基础上提出的 Lookahead 算法加入了前瞻性策略, 即任务分配时不仅考虑当前任务完成时间, 还对后续任务的完成时间进行预测, 综合考虑获得更优的决策, 但是 Lookahead 算法复杂度较高。Ilavarasan 等^[10]提出的 PETS 算法根据任务层次关系对任务进行分组, 同一分组中的任务可并行执行, 该算法复杂度较低, 但任务模型在异构环境中不具有普适性, 未考虑负载均衡的问题。

Fonseca 等^[11]在 2016 年提出一种分区并行任务的可调度分析方法, 最小化并行模型对底层上下文时序分析的影响, 显著地缩短了划分并行任务的最坏情况响应时间, 但只适用于特殊的处理器模型。

Pathan 等^[12]提出了一种两级抢占式全局固定优先级调度策略:任务级调度程序首先确定优先级最高的准备任务,然后子任务级调度程序选择其优先级最高的子任务执行。对于具有较大利用率的任务集,以及具有较高利用率任务数量的任务集具有得到较好的结果,但对于较低利用率任务数量的任务集效果较差。

刘少伟等^[13]针对 DAG 任务执行过程中资源利用率较低的问题,提出一种基于关键路径截取的有向无环图应用调度算法,该算法采取关键路径截取技术,循环找出最晚完成的未分配任务,从该任务出发,在所有未分配任务构成的图中找出最大连通子图,并计算该子图的关键路径,然后将关键路径上的任务集调度到性能匹配的虚拟机上执行同时通过任务回填技术提高资源使用率,但计算成本较高。

葛维春等^[14]针对 DAG 任务执行过程中负载均衡问题,提出了一种负载均衡优先的改进优先级表调度算法,算法以任务执行代价、任务的出度和任务间的通信代价作为参数计算任务优先级,并在任务调度时采用任务复制策略进一步优化调度过程。在考虑了通信开销的情况下,降低了 DAG 任务的完成时间并实现了良好的负载均衡性,但也增大了计算开销。

2020 年常爽等^[15]提出了一种新的多类型 DAG 任务转化(DTF)算法,在 DAG 重构基础上提出一个新的最坏响应时间分析方法,分析 DAG 任务执行的最坏响应时间,但是该算法在判断任务可调度性方面精确度较低,同时采用路径作为任务调度顺序的决定性因素,计算开销较大。

除了上述内容之外,陈刚等^[3]对 DAG 任务模型展开了讨论,将 DAG 调度算法分为 3 种:全局调度^[16-18]、联合调度^[19-21]和分解调度^[22]。其中,全局调度是将 DAG 任务直接由一个调度器来进行调度,调度的延迟随着总任务集利用率的提高而提高,但在选择哪个核进行调度为最佳情况时耗时较长,调度效果一般。分解调度是指将复杂的 DAG 任务转换成带有依赖关系的独立连续任务,再使用现有的调度算法进行调度。分解调度能够提高任务之间的并行性,适用性广易于在嵌入式设备上扩展。联合调度根据任务集的利用率划分为轻任务和重任务,轻任务仅需要在一个处理器核上顺序执行,重任务则需要多个处理器核才能满足其截止日期。联合调度算法灵活性更高,在其规定的条件下优于传统全局调度算法。不同类型任务调度算法在多核异构环境下所发挥出来的性能差异很大,具有很大的局限

性,因此研究高效且能在异构环境下充分发挥处理器性能的调度算法十分必要。

3 系统模型与相关定义

3.1 处理器模型

假设异构多核平台有 N 种不同类型的处理器,用 $C = \{C_1, C_2, \dots, C_N\}, C_n (n \in [1, N])$ 表示。每种处理器内核数量不完全相同, p_n 表示第 n 种类型处理器的内核数量, $p_n = |C_n|$ 。所有内核之间并行执行可进行数据通信,内核执行节点的时间单位均相同。

3.2 任务模型

异构多核系统调度的任务模型可以抽象为一个四元组 $G = \{T, E, R, c\}$ 。其中, T 表示 DAG 图的节点集合, $T = \{T_1, T_2, \dots, T_m\}$, T_i 表示任务图中第 i 个节点, $m = |T|$ 代表初始 DAG 的任务数量; E 表示 DAG 图的边的集合为二维矩阵, $e_{i,j} = k (e_{i,j} \in E)$ 表示节点 (T_i, T_j) 之间存在通信值为 k 的有向边。类型函数 $R: T \rightarrow C$ 代表 DAG 图中节点与处理器内核的对应关系,每个节点都有一种类型的处理器与之相对应,例如 $R(T_i) = n, (n \in [1, N])$ 表示节点 T_i 只能在类型为 C_n 的处理器资源上执行。权值函数 $c: T \times C$ 表示节点的最坏响应时间, $c(T_i)$ 表示节点 T_i 在其对应处理器内核 C_n 上可执行时间的最大值。

DAG 任务图实例 G 如图 1 所示。

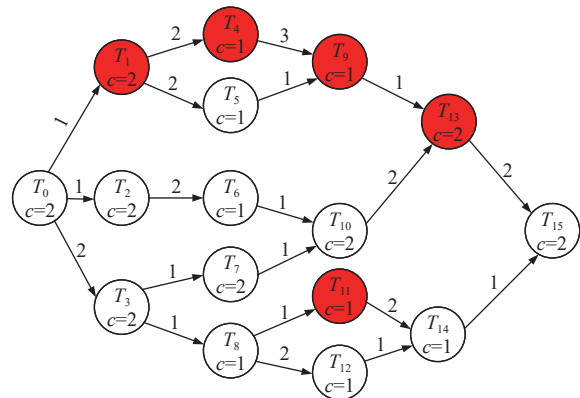


图 1 DAG 任务图实例 G

Fig. 1 DAG task map example G

定义 1 若存在 $e_{i,j} = 1 (e_{i,j} \in E)$, 那么称 T_i 为 T_j 的前驱节点, T_j 为 T_i 的后继节点, 即 T_j 只能在 T_i 执行后才能执行。pre(T_i) 和 suc(T_i) 分别表示 T_i 节点的前驱节点集合和后继节点集合。ans(T_i) 和 des(T_i) 表示节点 T_i 的祖先节点集合和子孙节点集合。

定义 2 将没有前驱的节点称为入口节点 T_{enter} , 没有后继的节点称为出口节点 T_{exit} , 为了不失一般性, 设每个图都有且只有一个入口节点和出口节点。

当存在多个入口节点(出口节点)时,可以在图中加入一个执行时间为 0 且通信边权值为 0 的入口/出口节点。

如果 T_i 和 T_j 被调度在不同的处理器核上,则边 (T_i, T_j) 的通信开销为 $|e_{i,j}|$, 用来表示任务之间通信开销大小。在同一处理器核上调度两父任务与子任务之间通信开销为 0。

$$\text{Trans}(T_i, T_j) = \begin{cases} 0 & T_i \rightarrow C_k \text{ and } T_j \rightarrow C_k \\ |e_{i,j}| & \text{other} \end{cases} \quad (1)$$

式(1)表示了任务之间通信开销的计算方法,如果在调度过程中不同内核之间频繁切换处理器,而且需要的通信数据量较大,通信的开销也会增加。为了减小这样的开销,本文提出一种重构复合节点的调度算法。

4 CNGA 算法

本文提出了 CNGA 算法。如果任务之间通信值较大,就把它们分配给相同的处理器核以减少通信开销。任务间处理器核切换次数越多带来的开销就越大,尽可能把同一路径上任务分配给同一处理器内核使任务间的通信值有效减少,与此同时还要避免造成处理器核负载过重。调度算法分为 3 个阶段:任务预处理、生成任务执行序列和处理器分配。

4.1 任务预处理

为了减小通信开销,把任务节点按照预处理划分后再分配到处理器内核上执行。

定义 3 复合节点:复合节点是包含一个或多个节点的子图,两个复合节点可以合并得到一个新

的复合节点。

从 DAG 任务图入口节点开始,每次都从节点为单位选择后继节点中同类型且通信值最大的边所连接节点进行合并生成复合节点。生成的复合节点中所包含的原子节点数要小于一个阈值 T_{Hold} 。 T_{Hold} 用来控制负载均衡,意义是处理器内核所负载的任务数量在一定范围内浮动。

$$T_{\text{Hold}} = \text{Max} \left(\left\lfloor \frac{|T_n|}{|C_n|} \right\rfloor, N_{\text{max}} \right) \times (1 + T_{\text{value}}), 0 \leq T_{\text{value}} \leq 1 \quad (2)$$

$$T_{\text{value}} = \frac{\text{当前节点的通信值}}{\text{当前节点所在局部关键路径上的总通信值}} \quad (3)$$

式中: $|T_n|$ 表示类型 n 的任务节点数量; $|C_n|$ 表示处理类型 n 任务节点所对应的处理器内核数量。 N_{max} 表示最大复合节点中的原子节点的数量; T_{value} 是一个百分比值,用来均衡负载和减小通信之间进行权衡。当 T_{value} 较大时,算法更倾向选择权值较大的边进行合并使通信量尽可能减小;当 T_{value} 较小时,权值较大的通信边可能会因为阈值不足而被放弃,此时算法更倾向于负载均衡。

图 2 为将原子节点合并为复合节点的示意图。图 3 为图 1 中 DAG 模型任务预处理之后转化生成的 CD-DAG 模型 G' 。经过预处理之后的任务图前驱后继关系不变,带箭头的虚线表示后继节点与前驱节点在同一处理器内核上,从而减少了通信开销。而 T_{Hold} 阈值约束使每个内核不会出现负载过重情况。

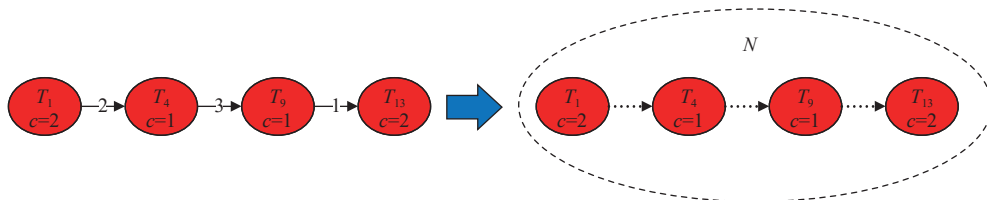


图 2 复合节点

Fig. 2 Composite node

4.2 优先级判定

在 DAG 图中,从入口节点 T_{enter} 开始执行到出口节点 T_{exit} 所经过的任务节点组成一条完整路径 path , 用 $\text{PATH} = \{\text{path}_1, \text{path}_2, \dots, \text{path}_k\}$ 表示图中所有的完整路径集合。所有路径中任务执行开销和通信开销之和最大的路径称为关键路径 $\text{CP}^{[10]}$ 。 $\text{len}(\text{path}_i)$ 表示调度完整路径 path_i 的时间开销, $\text{len}(G)$ 表示为调度完图 G 中关键路径 CP 的时间开销,位于 CP 路径上的任务执行与通信成本之和决定了最终调度长度的下限。

<http://www.journalmc.com>

$$\text{len}(\text{path}_k) = \sum_{T_i \in \text{path}_k} c(T_i) + \sum_{e_{i,j} \in \text{path}_k} |e_{i,j}| \quad (4)$$

$$\text{len}(\text{CP}) = \max_{\text{path}_k \in \text{PATH}} \{\text{len}(\text{path}_k)\} \quad (5)$$

定义 4 局部关键路径 DCP: 特定调度步骤期间,从当前节点到出口节点的最大路径称为局部关键路径,该路径是当前任务执行开销和通信开销最大的路径。

图 3 中每个节点代表一个任务,任务节点名称和任务节点开销在图中给出。不同颜色表示任务节点需要在不同种类的处理器内核上去执行。 $\text{pre}(T_0) =$

$\emptyset$, $\text{succ}(T_{15}) = \emptyset$, 则 $T_{\text{netter}} = T_0$, $T_{\text{exit}} = T_{15}$ 。

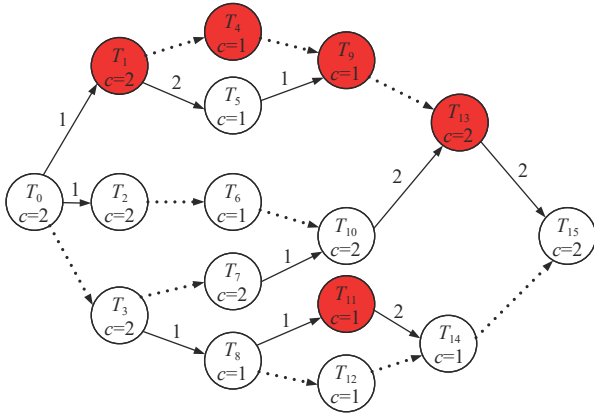


图3 预处理后 G'

Fig. 3 After pretreatment G'

图3中的完整路径如表1所示。

表1 完整路径列表

Tab. 1 Full path list

路径	节点集合	长度
path ₁	$\{T_0, T_1, T_4, T_9, T_{13}, T_{15}\}$	13
path ₂	$\{T_0, T_1, T_5, T_9, T_{13}, T_{15}\}$	16
path ₃	$\{T_0, T_2, T_6, T_{10}, T_{13}, T_{15}\}$	16
path ₄	$\{T_0, T_3, T_7, T_{10}, T_{13}, T_{15}\}$	17
path ₅	$\{T_0, T_3, T_8, T_{11}, T_{14}, T_{15}\}$	13
path ₆	$\{T_0, T_3, T_8, T_{12}, T_{14}, T_{15}\}$	10

则关键路径 $CP = \text{path}_4 = \{T_0, T_3, T_7, T_{10}, T_{13}, T_{15}\}$ 。

优先级判定能够直接影响调度目标的选取, 从而影响任务的最坏响应时间。如果全部任务能够尽早分配处理器核上执行并使得整体响应时间减少, 那应该给予该任务高优先级。CNGA 算法将根据节点对于后续节点任务影响的程度作为优先级判定的参数, 并按照由大到小的顺序进行排序。主要分为两步:

(1) 当前节点 T_i 到出口节点 T_{exit} 的局部关键路径长度 L_{DCP} , 表示当前节点到出口节点构成的子图的最坏响应时间的下限。 L_{DCP} 越长证明当前节点的子孙节点越多, 对后续节点的影响越大。 L_{DCP} 所以作为决定任务优先级的主要参数, L_{DCP} 越长优先级越高。

(2) 当前节点 T_i 到出口节点 T_{exit} 的局部关键路径上的通信时间 $E_{i,\text{exit}}$, 表示当前节点到出口节点的子图用于处理器内核调度的通信时间。 $E_{i,\text{exit}}$ 越大证明用于处理器内核调度的通信时间越长, 则用于任务执行的占比越少。则 $E_{i,\text{exit}}$ 作为决定任务优先级的次

要参数, $E_{i,\text{exit}}$ 越小优先级越高。

$$E_{i,\text{exit}} = \sum_{n=i}^{\text{exit}} e_{n,n+1} \quad (6)$$

$$\mu = \frac{E_{i,\text{exit}}}{L_{\text{DCP}}} \quad (7)$$

式中: μ 表示从当前节点 T_i 到出口节点 T_{exit} 通信时间在关键路径长度的占比。

综上所述, 任务调度优先级的判定方式为

$$\text{priority}(T_i) = L_{\text{DCP}} - \mu \quad (8)$$

4.3 生成任务执行序列

表2为根据 $\text{priority}(T_i)$ 判定得到的任务优先级列表。

表2 任务优先级列表

Tab. 2 Task priority list

节点名称	$L_{\text{DCP}}(T_i)$	$\mu(T_i)$	$\text{priority}(T_i)$
T_0	16	0.312 5	15.687 5
T_1	13	0.384 6	12.614 5
T_2	13	0.307 7	12.692 3
T_3	15	0.333 3	14.666 7
T_4	8	0.250 0	7.750 0
T_5	9	0.333 3	8.666 7
T_6	11	0.363 6	10.636 4
T_7	13	0.384 6	12.615 4
T_8	8	0.375 0	7.625 0
T_9	7	0.285 7	6.713 4
T_{10}	10	0.400 0	9.600 0
T_{11}	6	0.333 3	5.666 7
T_{12}	4	0.000 0	4.000 0
T_{13}	6	0.333 3	5.666 7
T_{14}	3	0.000 0	3.000 0
T_{15}	2	0.000 0	2.000 0

通过表2可得, 本文提出的优先级判定方式不但考虑到每个任务对于后续任务的影响, 而且均衡通信时间占比, 避免了过多时间浪费在内核通信上。

所生成的任务序列分别为

$$C_1 = \{T_0, T_3, T_2, T_7, T_6, T_{10}, T_5, T_8, T_{12}, T_{14}, T_{15}\}$$

$$C_2 = \{T_1, T_4, T_9, T_{11}, T_{13}\}$$

在表2所示的 DAG 列表任务优先级的基础上, 进行如下处理:

步骤1 若 DAG 图有多个起始(终止)节点, 则设立一个虚拟节点作为起始(终止)节点。

步骤 2 进行节点合并生成复合节点, 从而减少通信开销, 得到转换后的 G' 。

步骤 3 从序列中获取入度为 0 的节点, 并组成就绪节点集合, 执行步骤 4。

步骤 4 将就绪节点集按照优先级的大小依次分配给空闲的处理器内核, 优先分配优先级较大的节点。

步骤 5 若任务节点的优先级大小相同, 则优先分配局部关键路径长的节点。

步骤 6 若节点的局部关键路径长度相同, 则优先分配直接后继节点通信值之和较小的节点。

步骤 7 若上述步骤依旧均无法分配节点, 则按照节点序列的顺序分配节点。

步骤 8 重复步骤 1~步骤 6, 直至所有节点执行结束。

CNGA 调度算法伪代码如下:

Algorithm: CNGA

输入: DAG图 $G = \{T, E, R, c\}$, 异构处理器内核

输出: 所有节点调度结果

1. 规范化 DAG 图, 初始节点不唯一时设置虚拟节点;
 2. WHILE($T_i \in G'$)
 3. IF($\exists T_{i\text{type}} = T_{j\text{type}}$ 且复合节点满足阈值)
 4. 选取 T_i 后继同类型节点中通信值最大的进行合并生成复合节点;
 5. ELSE
 6. 跳转到 2;
 7. 递归计算每个节点到出口节点的局部关键路径 L_{DCP} 和子孙节点的通信开销 $E_{i,\text{exit}}$;
 8. 根据优先级判定方式计算优先级 $\text{priority}(T_i)$;
 9. WHILE(仍有未调度的 DAG) do
 10. IF(当前时刻有新的 DAG 到达)
 11. 遍历当前所有参与调度的 DAG 并跳转到 1;
 12. ELSE
 13. 根据算法选择优先级最高的任务节点, 为该任务分配处理器内核并开始调度执行;
 14. END IF
 15. IF(当前时刻有节点完成调度) do
 16. 将此节点在 DAG 任务中删除, 并输出相关调度结果;
 17. END IF
 18. END WHILE
 19. RETURN 所有节点调度结果;
-

<http://www.journalmc.com>

算法第 1 行对算法输入的 DAG 任务图的所有节点进行了初始化操作, 同时初始化复合节点。第 2~6 行对原有 DAG 任务图进行预处理, 减少不必要的通信值。第 7~8 行通过式(8)计算每个节点的优先级。第 9~19 行获取当前的就绪任务列表, 按照优先级对就绪的任务列表进行排序。并在 DAG 中调度过后删除对应的节点, 直到没有符合要求的节点存在或当前节点无对应的空闲处理器内核。时间复杂度为 $O(|T|^2)$, 空闲复杂度为 $O(|T'|)$ 。

4.4 处理器模拟调度

图 1 中 DAG 任务分别在 HEFT 算法和 CNGA 算法下的调度情况如图 4 所示。可以看出, CNGA 算法的调度响应时间小于 HEFT 算法的, 而 CNGA 算法在调度过程中 3 个处理器核的空闲时间也明显小于 HEFT 算法的。

本算法很好地平衡了内核通信开销和内核负载过重的情况, 通过复合节点的方式减少通信开销对于任务到内核的分配使内核间通信和任务执行的总开销尽可能小, 从而减少整体任务的响应时间。

5 实验分析

在 4.4 小节已经针对前文中给定 DAG 案例进行了对比, 图 4 能够明显看出 HEFT 和 CNGA 算法的调度过程和结果。为了达到相应的对比效果验证 CNGA 算法的性能, 在这一小节将对随机生成的 DAG 任务图进行调度, 分别与 HEFT、CPOP 算法进行对比^[23]。

在仿真实验中, 通过设置一系列参数来进行 DAG 结构的控制。以下是给出的参数及其含义。

- n : DAG 任务数量, 在 [20, 50] 之间选取。
- K : 处理器类型。数量在 [2, 10] 范围内随机选取, 每种处理器的内核个数 C_k 在 [2, 8] 中随机选取。
- fat : 同层 DAG 内节点数量上限, 在 $n \times \{0.2, 0.4, 0.8\}$ 之中选取。
- CCR : 任务传输量与任务计算量之间的比值, 在 $\{0.1, 0.5, 0.8, 1.0, 2.0, 5.0\}$ 之间选取。
- P : 两个节点存在边的概率。
- U : 任务的总利用率。通过 UnniFast 方法^[23]对所有任务节点的任务利用率进行随机分配, $U = \sum_{T_i \in G} u(T_i)$, 获取所有任务节点的最坏响应时间, $u(T_i)$ 为节点 T_i 的利用率。任务总利用率 U 从 [0.5, 5.0] 中随机选取, 并且 DAG 任务所有节点的最坏执行时间之和为 $\text{vol}(G) = U \times T$, 其中截止时间 $D \leq T$ 。

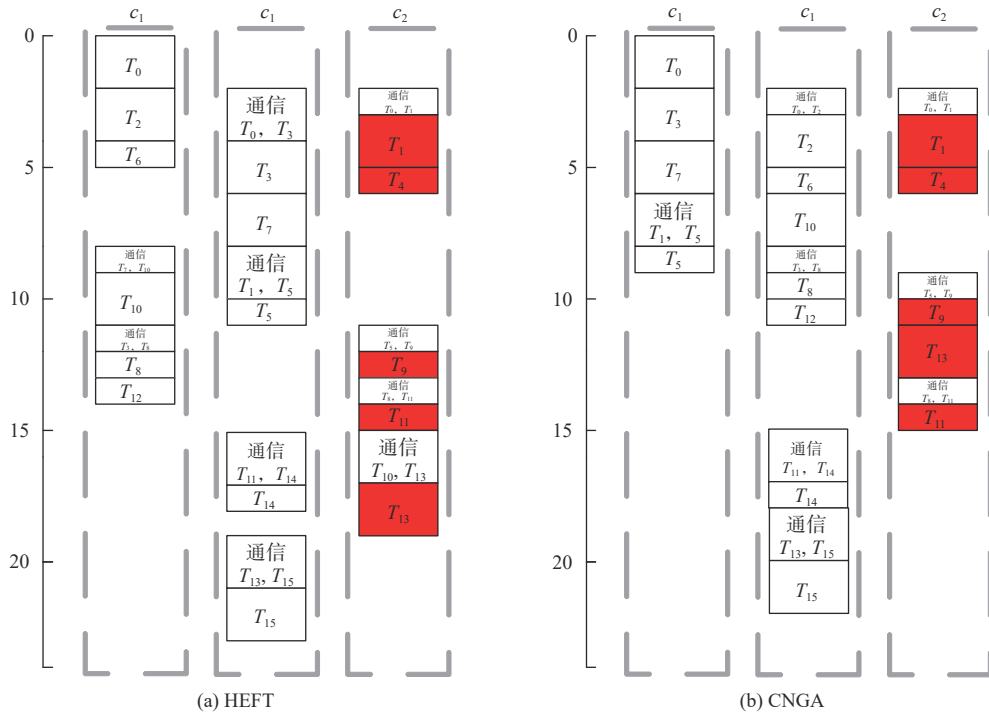


图4 调度过程

Fig. 4 Scheduling process

此外, 根据 Makespan 来衡量各个算法的调度性能。

$$\text{Makespan} = \max \{ \text{ATF}(T_{\text{exit}}) \} \quad (9)$$

式中: $\text{ATF}(T_i)$ 表示节点 T_i 的最晚完成时间。

通过实验仿真获取各算法在不同情况下的 Makespan。并使用以下指标对结果进行比较分析:

(1) 接受率(Acceptance Ratio, 记为 P_{AR}): 指定最坏响应时间上界判定为可调度的任务数与生成的总任务数之间的比率。接受率越高, 表明该算法的最坏响应时越精确。

$$P_{\text{AR}} = \sum_{G_i \in \text{set}} \frac{\text{count}(\text{Makespan}(G_i) < D_i)}{|\text{set}|} \quad (10)$$

式中: set 为 DAG 任务集; $\text{Makespan}(G_i)$ 为第 i 个任务图的最坏响应时间; D_i 为当前任务图 G_i 的截止日期; $|\text{set}|$ 表示当前任务集中的数据量。

(2) 加速比(Speedup, 记为 R_{Speedup}): 任务顺序执行时间与最坏响应时间的比值, 可获得当前算法对于一个任务调度的加速情况。

$$R_{\text{Speedup}} = \frac{\text{vol}(G)}{\text{Makespan}} \quad (11)$$

式中: $\text{vol}(G)$ 表示任务顺序执行的时间开销。为防止偶然性, 去多组数据取平均值。

(3) 松弛度(Slack, 记为 S): Slack 是一个关于任务调度算法鲁棒性的量度, 其反映的是一个算法

调度产生的任务最坏响应时间的不确定程度。Slack 的定义如式(12)所示:

$$S = \frac{n}{\sum_{i=1}^n \text{Makespan} - \text{Enter}(T_i) - \text{Exit}(T_i)} \quad (12)$$

式中: n 为任务节点的数量; $\text{Enter}(T_i)$ 表示入口节点 T_{enter} 到任务节点 T_i (不包含任务 T_i) 最长路径的长度; $\text{Exit}(T_i)$ 表示任务节点 T_i 到终止节点 T_{exit} 最长路径的长度。

从实验结果中可以看出, 随着 DAG 任务数量的增加, 任务之间处理器竞争越来越激烈。如图 5 所示, CNGA 算法在不同任务数量下 Makespan 相比 HEFT 算法的提升 4.81% ~ 15.57%, 相比 CPOP 算法的提升 3.50% ~ 10.40%。如图 6 所示, CNGA 算法在不同任务数量下加速比相比 HEFT 算法的平均提高 8.73%。

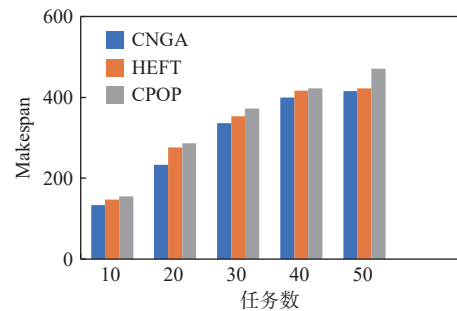


图5 Makespan 随任务数变化情况

Fig. 5 Makespan with number of tasks

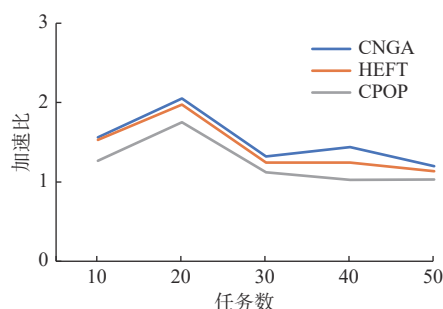


图6 加速比随任务数变化情况

Fig. 6 Speedup with number of tasks

松弛度和接受率两个参数在一定程度上代表了算法在调度过程中的稳定性, CNGA 算法的接受率稳定高于 HEFT 和 CPOP 算法的。如图 7 和图 8 所示, 松弛度和接受率在相同情况下均优于 HEFT 和 CPOP 算法的, CNGA 算法在稳定性上相比 HEFT 和 CPOP 算法的平均提高了 3.96%。综合上述内容, 本文提出的 CNGA 算法与现有算法相比, 各方面性能都获得较大提升。

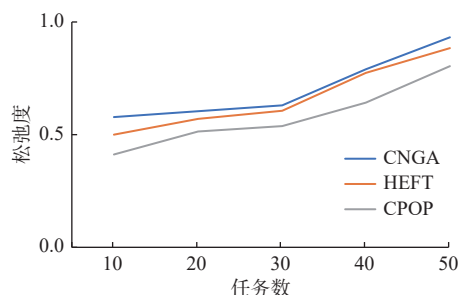


图7 松弛度随任务数变化情况

Fig. 7 Slack with number of tasks

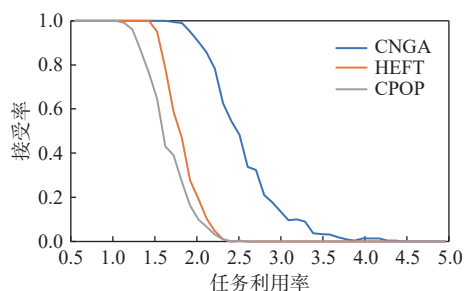


图8 接受率随任务利用率变化情况

Fig. 8 Acceptance ratio with number of tasks

6 结束语

本文对异构多核平台下具有通信开销的 DAG 任务调度问题进行了研究, 在综合考虑通信开销计算和处理器核分配的问题上提出了基于复合节点的 CNGA 算法。该算法在尽可能减小通信开销的同时, <http://www.journalmc.com>

用限制复合节点大小的方法兼顾了负载均衡。实验结果表明, 该算法具有良好的调度性能, 能够很好发挥出异构多核处理器的优势。

参考文献:

- [1] ULLMAN J D. NP-complete scheduling problems[J]. *Journal of Computer and System Sciences*, 1975, 10(3): 384-393. DOI: 10.1016/S0022-0000(75)80008-0.
- [2] 周楠, 胡娟, 胡海明. 多核处理器发展趋势及关键技术[J]. *计算机工程与设计*, 2018, 39(2): 393-399. DOI: 10.16208/j.issn1000-7024.2018.02.018.
- ZHOU N, HU J, HU H M. Development trend and key techniques of multi-core processor[J]. *Computer Engineering and Design*, 2018, 39(2): 393-399. DOI: 10.16208/j.issn1000-7024.2018.02.018.
- [3] 陈刚, 关楠, 吕鸣松, 等. 实时多核嵌入式系统研究综述[J]. *软件学报*, 2018, 29(7): 2152-2176. DOI: 10.13328/j.cnki.jos.005580.
- CHEN G, GUAN N, LYU M S, et al. State-of-the-art survey of real-time multicore system[J]. *Journal of Software*, 2018, 29(7): 2152-2176. DOI: 10.13328/j.cnki.jos.005580.
- [4] TOPCUOGLU H, HARIRI S, WU M Y. Performance-effective and low-complexity task scheduling for heterogeneous computing[J]. *IEEE Transactions on Parallel and Distributed Systems*, 2002, 13(3): 260-274. DOI: 10.1109/71.993206.
- [5] 潘燕萍. 异构多核处理器的微内核操作系统通信机制的研究与设计[D]. 桂林: 桂林理工大学, 2018. DOI: 10.27050/d.cnki.gglgc.2018.000107.
- PAN Y P. Research and design of the communication mechanism of microkernel operating system for heterogeneous multicore processors[D]. Guilin: Guilin University of Technology, 2018. DOI: 10.27050/d.cnki.gglgc.2018.000107.
- [6] 安建峰, 游红俊, 赵伟勋, 等. 星载异构计算环境下的能耗优化任务调度[J]. *上海航天(中英文)*, 2021, 38(4): 38-44. DOI: 10.19328/j.cnki.2096-8655.2021.04.006.
- AN J F, YOU H J, ZHAO W X, et al. Real-time task scheduling for space-oriented computing on heterogeneous system[J]. *Aerospace Shanghai (Chinese & English)*, 2021, 38(4): 38-44. DOI: 10.19328/j.cnki.2096-8655.2021.04.006.
- [7] MUNIR E U, MOHSIN S, HUSSAIN A, et al. SDBATS: a novel algorithm for task scheduling in heterogeneous computing systems[C]//Proceedings of 2013 IEEE International Symposium on Parallel & Distributed Processing, Workshops and Phd Forum. Piscataway: IEEE, 2013: 43-53. DOI: 10.1109/IPDPSW.2013.259.
- [8] WANG G, GUO H, WANG Y X. A novel heterogeneous

- scheduling algorithm with improved task priority[C]//Proceedings of 2015 IEEE 17th International Conference on High Performance Computing and Communications, 2015 IEEE 7th International Symposium on Cyberspace Safety and Security, and 2015 IEEE 12th International Conference on Embedded Software and Systems. Piscataway: IEEE, 2015: 1826-1831. DOI: [10.1109/HPCC-CSS-ICSS.2015.48](https://doi.org/10.1109/HPCC-CSS-ICSS.2015.48).
- [9] BITTENCOURT L F, SAKELLARIOU R, MADEIRA E R M. DAG scheduling using a lookahead variant of the heterogeneous earliest finish time algorithm[C]//Proceedings of the 18th Euromicro Conference on Parallel, Distributed and Network-based Processing. Piscataway: IEEE, 2010: 27-34. DOI: [10.1109/PDP.2010.56](https://doi.org/10.1109/PDP.2010.56).
- [10] ILAVARASAN E, THAMBIDURAI P, MAHILMAN-NAN R. Performance effective task scheduling algorithm for heterogeneous computing system[C]//Proceedings of the 4th International Symposium on Parallel and Distributed Computing. Piscataway: IEEE, 2005: 28-38. DOI: [10.1109/ISPD.2005.39](https://doi.org/10.1109/ISPD.2005.39).
- [11] FONSECA J, NELISSEN G, NELIS V, et al. Response time analysis of sporadic DAG tasks under partitioned scheduling[C]//Proceedings of the 11th IEEE Symposium on Industrial Embedded Systems. Piscataway: IEEE, 2016: 1-10. DOI: [10.1109/SIES.2016.7509443](https://doi.org/10.1109/SIES.2016.7509443).
- [12] PATHAN R, VOUDOURIS P, STENSTRÖM P. Scheduling parallel real-time recurrent tasks on multicore platforms[J]. *IEEE Transactions on Parallel and Distributed Systems*, 2018, 29(4): 915-928. DOI: [10.1109/TPDS.2017.2777449](https://doi.org/10.1109/TPDS.2017.2777449).
- [13] 刘少伟, 任开军, 邓科峰, 等. 云平台上基于关键路径截取的有向无环图应用调度算法[J]. *国防科技大学学报*, 2017, 39(3): 97-104. DOI: [10.11887/j.cn.201703016](https://doi.org/10.11887/j.cn.201703016).
LIU S W, REN K J, DENG K F, et al. Directed acyclic graph application scheduling strategy based on critical path cut on cloud platform[J]. *Journal of National University of Defense Technology*, 2017, 39(3): 97-104. DOI: [10.11887/j.cn.201703016](https://doi.org/10.11887/j.cn.201703016).
- [14] 葛维春, 叶波. 负载均衡优先的改进优先级表调度算法[J]. *沈阳工业大学学报*, 2017, 39(3): 241-247. DOI: [10.7688/j.issn.1000-1646.2017.03.01](https://doi.org/10.7688/j.issn.1000-1646.2017.03.01).
GE W C, YE B. List scheduling algorithm of improved priority with considering load balance[J]. *Journal of Shenyang University of Technology*, 2017, 39(3): 241-247. DOI: [10.7688/j.issn.1000-1646.2017.03.01](https://doi.org/10.7688/j.issn.1000-1646.2017.03.01).
- [15] 常爽爽, 赵栩锋, 刘震宇, 等. 基于异构多核的多类型 DAG 任务的响应时间分析[J]. *计算机学报*, 2020, 43(6): 1052-1068. DOI: [10.11897/SP.J.1016.2020.01052](https://doi.org/10.11897/SP.J.1016.2020.01052).
- CHANG S S, ZHAO Y F, LIU Z Y, et al. Response time analysis of typed DAG tasks on heterogeneous multi-cores[J]. *Chinese Journal of Computers*, 2020, 43(6): 1052-1068. DOI: [10.11897/SP.J.1016.2020.01052](https://doi.org/10.11897/SP.J.1016.2020.01052).
- [16] BONIFACI V, MARCHETTI-SPACCAMELA A, STILLER S, et al. Feasibility analysis in the sporadic DAG task model[C]//Proceedings of the 25th Euromicro Conference on Real-Time Systems. Piscataway: IEEE, 2013: 225-233. DOI: [10.1109/ECRTS.2013.32](https://doi.org/10.1109/ECRTS.2013.32).
- [17] BARUAH S. Improved multiprocessor global schedulability analysis of sporadic DAG task systems[C]//Proceedings of the 26th Euromicro Conference on Real-Time Systems. Piscataway: IEEE, 2014: 97-105. DOI: [10.1109/ECRTS.2014.22](https://doi.org/10.1109/ECRTS.2014.22).
- [18] MELANI A, BERTOONA M, BONIFACI V, et al. Response-time analysis of conditional DAG tasks in multiprocessor systems[C]//Proceedings of the 27th Euromicro Conference on Real-Time Systems. Piscataway: IEEE, 2015: 211-221. DOI: [10.1109/ECRTS.2015.26](https://doi.org/10.1109/ECRTS.2015.26).
- [19] LI J, CHEN J J, AGRAWAL K, et al. Analysis of federated and global scheduling for parallel real-time tasks[C]//Proceedings of the 26th Euromicro Conference on Real-Time Systems. Piscataway: IEEE, 2014: 85-96. DOI: [10.1109/ECRTS.2014.23](https://doi.org/10.1109/ECRTS.2014.23).
- [20] BARUAH S. Federated scheduling of sporadic DAG task systems[C]//Proceedings of 2015 IEEE International Parallel and Distributed Processing Symposium. Piscataway: IEEE, 2015: 179-186. DOI: [10.1109/IPDPS.2015.33](https://doi.org/10.1109/IPDPS.2015.33).
- [21] LEUNG J Y T, LI C L. Scheduling with processing set restrictions: a survey[J]. *International Journal of Production Economics*, 2008, 116(2): 251-262. DOI: [10.1016/j.ijpe.2008.09.003](https://doi.org/10.1016/j.ijpe.2008.09.003).
- [22] LEUNG J Y T, LI C L. Scheduling with processing set restrictions: a literature update[J]. *International Journal of Production Economics*, 2016, 175: 1-11. DOI: [10.1016/j.ijpe.2014.09.038](https://doi.org/10.1016/j.ijpe.2014.09.038).
- [23] BINI E, BUTTAZZO G C. Measuring the performance of schedulability tests[J]. *Real-Time Systems*, 2005, 30(1/2): 129-154. DOI: [10.1007/s11241-005-0507-9](https://doi.org/10.1007/s11241-005-0507-9).

作者简介:

郝鹏涛 硕士研究生, haopengtao@st.xatu.edu.cn

肖 锋(通信作者) 教授, 硕士生导师,

xffriends207@163.com